

## 5. Evaluation and Analysis

The full-stack web application developed for this assignment is a simple RTA Merchant Upload Batch File Portal.

It demonstrates how an Angular front-end and a Spring Boot back-end work together to implement the core functionality of a production-style business system.

Each implemented feature supports the business logic expected in my FYP 2 deliverables, allowing merchants to log in, upload and delete batch files, check activity logs and manage their personal information.

### a) Choice of API endpoints and parameters

This full-stack web application includes operations such as login, profile photo upload, update profile information, and batch file upload.

Each endpoint follows RESTful conventions using nouns and hierarchical paths, such as `/api/auth/login`, `/api/profile/{merchantId}`, and `/api/batches/upload`.

This naming pattern allows the front-end to easily map user actions (login, profile update, file upload, delete) to their corresponding back-end logic.

The use of path variables (for example, `merchantId`) provides a clean, resource-oriented design that scales easily when new entities, such as merchant keys, are added in FYP 2.

### b) Choice of components and routes

The front-end is divided into standalone components that represent main business modules:

- **LoginComponent** handles user authentication.
- **BatchListComponent** manages file uploads and batch listings.
- **ViewProfileComponent** displays and edits merchant information.

The routing configuration in `app.routes.ts` provides clear navigation paths (`/login`, `/batch-list`, `/profile`) with `AuthGuard` protection to prevent unauthorized access.

This modular routing structure makes it easy to expand the system later, for example by adding new merchant or analytics pages.

### c) Use of bindings and directives

Angular interpolation (`{{ }}`) and event bindings (`(click)`, `(ngSubmit)`) were used to make the interface dynamic and responsive.

Conditional rendering (`*ngIf`) and list rendering (`*ngFor`) simplify page updates when data changes.

Template-driven and reactive forms give users instant feedback on input validation. These bindings ensure that the user interface immediately reflects updates from the back-end without reloading the page.

#### **d) HTTP client calls placement**

All HTTP requests are handled in dedicated service classes — `AuthService`, `PortalService`, and `ProfileService` — instead of being written directly in components. This separation of concerns improves maintainability and testability by keeping components focused on the interface and user interaction, while services handle data communication and error handling.

It also makes the system easier to update in the future when new endpoints are added in FYP 2.

#### **e) Form design and validation**

Template-driven and reactive forms were implemented in the Login and Profile modules to collect user input efficiently.

Basic validation ensures that empty or invalid fields are detected immediately, improving user experience and reducing server-side validation.

This design supports the FYP 2 goal of ensuring data accuracy and providing instant user feedback before submission.

#### **f) Relation to end-user UI and business logic**

From an end-user point of view, the system offers a simple and secure workflow: merchants log in, upload batch files, and manage their profiles smoothly without confusion or error.

Each action on the interface directly triggers a REST API operation on the back-end — for example, when a file is uploaded, both the database and the activity log update immediately.

This close connection between the front-end and back-end ensures that the system is intuitive, efficient, and scalable for future modules such as analytics or automation in FYP 2.

## Angular Features Checklist

| No | Feature                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Dynamic page changes via interpolation and property binding with appropriate template expressions                                                                                                                                                                                                                                                                                                                                                  |
|    | Implementation:<br>1. batch-list.component.ts<br>Uses interpolation to display data such as {{merchant?.name}}, {{batch.fileName}}, {{batch.status}} etc.<br><br>2. view-profile.component.ts<br>Displays user information using bindings {{profile.name}}, {{profile.email}}, {{profile.company}}                                                                                                                                                 |
| 2  | Dynamic page changes via class and style binding and/or NgClass and NgStyle together with basic CSS style rules                                                                                                                                                                                                                                                                                                                                    |
|    | Implementation:<br>1. batch-list.component.ts & view-profile.component.ts<br>Apply CSS styling and dynamic UI effects (e.g., hover states, active highlighting) to reflect real-time visual changes.                                                                                                                                                                                                                                               |
| 3  | Event binding to respond to user events on web page                                                                                                                                                                                                                                                                                                                                                                                                |
|    | Implementation:<br><br>1. Batch-list.component.ts<br>Event handlers (click)="uploadBatch()", (click)="deleteBatch(batch.id!)", (change)="onFileSelected(\$event)" handle file uploads and actions.<br><br>2. View-profile.component.ts<br>Uses (click)="openPhotoUpload()", (change)="onFileSelected(\$event)", (click)="updateProfile()" for profile updates.<br><br>3. login.component.ts<br>Handles form submission via (ngSubmit)="onLogin()". |
| 4  | App should be structured as a component hierarchy consisting of root, child, grand-child components                                                                                                                                                                                                                                                                                                                                                |
|    | Implementation:<br><br>app.component.ts = root component<br><br>login.component.ts, batch-list.component.ts, view-profile.component.ts = child components registered within app.routes.ts.                                                                                                                                                                                                                                                         |
| 5  | Conditional dynamic page changes using @if and @else or @switch and @case directives                                                                                                                                                                                                                                                                                                                                                               |
|    | Implementation:                                                                                                                                                                                                                                                                                                                                                                                                                                    |

|   |                                                                                                                                                                                                                                                                                                    |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | <p>1. View-profile.component.ts<br/>Uses *ngIf="profile" and *ngIf="profile.profilePhotoUrl" to control UI rendering based on data presence.</p> <p>2. Login.component.ts<br/>Displays error messages conditionally: &lt;p *ngIf="errorMessage" class="error"&gt;{{ errorMessage }}&lt;/p&gt;.</p> |
| 6 | Rendering of multiple HTML elements and/or child components using @for. Include appropriate use of loop contextual variables \$index, \$count, \$first.                                                                                                                                            |
|   | <p>Implementation:</p> <p>1. batch-list.component.ts<br/>Iterates over batches and activityLogs using *ngFor="let batch of batches" and *ngFor="let log of activityLogs".</p>                                                                                                                      |
| 7 | Create reactive forms with multiple grouped form control elements to accept user input                                                                                                                                                                                                             |
|   | <p>Implementation:</p> <p>1. Login.component.ts<br/>Implements a template-driven form using [(ngModel)]="username" and [(ngModel)]="password"</p> <p>2. View-profile.component.ts<br/>Uses a grouped form (#editForm="ngForm") for editing company, contact, and address fields.</p>               |
| 8 | Implement validation of form controls as well as appropriate display of validation error messages                                                                                                                                                                                                  |
|   | <p>Implementation:</p> <p>1. login.component.ts<br/>Validates empty inputs and displays an error message via errorMessage binding.</p> <p>2. view-profile.component.ts<br/>Uses &lt;input required&gt; for validation and provides user feedback through alerts.</p>                               |
| 9 | Use HTTPClient library to send standard HTTP requests                                                                                                                                                                                                                                              |
|   | <p>Implementation:</p> <p>1. auth.service.ts<br/>login() sends a POST request to /api/auth/login.</p> <p>2. portal.service.ts<br/>Implements getBatches(), uploadBatch(), deleteBatch() (GET / POST / DELETE)</p> <p>3. profile.service.ts</p>                                                     |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | Handles fetchProfile(), updateProfile(), uploadProfilePhoto() (GET / PUT / POST).                                                                                                                                                                                                                                                                                                                                                                           |
| 10 | <p>Registering and matching routes to components with appropriate URL path mappings, including route redirecting and wildcard routes.</p> <p>Implementation:</p> <ol style="list-style-type: none"> <li>1. app.routes.ts<br/>Defines:<br/>{ path: 'login' ... },<br/>{ path: 'batch-list' ... },<br/>{ path: '', redirectTo: '/login', pathMatch: 'full' }</li> <li>2. app.routes.server.ts<br/>{ path: '**', renderMode: RenderMode.Prerender }</li> </ol> |
| 11 | <p>Implementing programmatic navigation to routes where appropriate</p> <p>Implementation:</p> <ol style="list-style-type: none"> <li>1. login.component.ts<br/>Uses this.router.navigate(['/batch-list']) for navigation after successful login.</li> </ol>                                                                                                                                                                                                |
| 12 | <p>Provide short comments</p> <p>Implementation:</p>                                                                                                                                                                                                                                                                                                                                                                                                        |

## Spring Boot Backend Features Checklist

| No | Feature                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <p>RESTful API endpoint design</p> <p>Implementation:</p> <ol style="list-style-type: none"> <li>1. AuthController.java<br/>Implements /api/auth/login endpoint for user authentication.</li> <li>2. ProfileController.java<br/>Provides /api/profile/{merchantId} (GET/PUT) for profile retrieval and update, and /api/profile/{merchantId}/photo (POST) for uploading profile photos.</li> <li>3. RtaBatchController.java<br/>/api/batches (GET all),<br/>/api/batches/upload (POST upload new batch),<br/>/api/batches/{id} (DELETE by ID),<br/>/api/batches/{id}/process (POST for batch processing).</li> </ol> |
| 2  | <p>Spring Starter + Annotations</p> <p>Implementation:</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | <p>1. SecurityConfig.java<br/>Uses @Configuration, @EnableWebSecurity, and @Bean to configure CORS and security settings.</p> <p>2. WebConfig.java<br/>Uses @Configuration and @Bean for global CORS configuration.</p> <p>3. AuthController.java, ProfileController.java, RtaBatchController.java<br/>Annotated with @RestController, @CrossOrigin, @RequestMapping, and @Autowired for dependency injection and endpoint registration.</p> <p>4. ProfileRepository, RtaBatchRepository, RtaTransactionRepository<br/>Annotated with @Repository and extend JpaRepository to leverage Spring Data JPA for CRUD operations.</p>                                  |
| 3 | <p>Domain classes + DB schema</p> <p>Implementation:</p> <p>Domain Classes:</p> <p>1. RtaBatch.java<br/>Represents the rta_batches table; includes fields such as fileName, merchantId, status, createdAt.</p> <p>2. MerchantProfile.java<br/>Represents the profiles table; includes fields such as merchantId, name, email, company, contact, joinedOn, profilePhotoUrl.<br/>Profiles</p> <p>SQL Schema:</p> <p>V1__Create_initial_schema.sql - Creates the rta_batches and rta_transactions table</p> <p>V2__Create_profile_table.sql - Creates the profiles table</p>                                                                                        |
| 4 | <p>CRUD Operations</p> <p>Implementation:</p> <p>1. RtaBatchController.java</p> <ul style="list-style-type: none"> <li>• getAllBatches() → @GetMapping (Retrieve all records)</li> <li>• uploadBatch() → @PostMapping (Create new batch)</li> <li>• deleteBatch() → @DeleteMapping (Remove a record by ID)</li> </ul> <p>2. ProfileController.java → getProfileByMerchantId() (GET one)</p> <ul style="list-style-type: none"> <li>• getProfileByMerchantId() → @GetMapping (Retrieve a single profile)</li> <li>• updateProfile() → @PutMapping (Update profile information)</li> <li>• uploadProfilePhoto() → @PostMapping (Upload a profile photo)</li> </ul> |